

StarHub 实时视频陪伴教程

专注模式 · 几乎事无巨细版

上一份教材我们把最基本的记忆功能搭好了。这一份做实时视频的功能，最终实现的状态是：你的机能一边真的看着你、一边跟你说话。

目标栈：Python 3.11+ / SQLite / 任意 LLM API

所有代码为参考实现，允许适配；所有 **MUST** / **NEVER** 条款不允许违反

🔗 结论：

此教程真正新增的东西只有三样：一双眼睛、一对流式耳朵和嘴、一个仲裁器。其余全是上一份教程的复用。为什么能这么省？因为上份教程那个架构早就给你留好了口子，在第 2 章会讲清楚，建议人类可以先看个大概，心里有个数

----- ✦ *🔗* ✦ -----

🔗 本教程的边界，也说在前面：

1 写死的部分：呈现非常具体的架构和方案，包括最难的架构陷阱（我自己被坑过的地方）：状态机、协议字段、边界条件

2 没写死的部分：此教程并非全量源代码包，留白的部分可以让你的机有自由空间提出更好的、适配你们部署环境的实施细节

3 我无法提供的、必须由你们自己写的部分：麦克风采集与重采样、音频工作线程、摄像头取帧与编码、浏览器与服务器之间的消息协议、鉴权与加密、背压与重连、各家 API 具体调用、依赖清单与启动命令，需要你的机根据你们具体部署环境调配。

4 诚实地说：架构实际落地过程中也来回修改多次，并且目前依旧有优化空间，但是本教程可以保证生成结构正确、状态机不会卡死、不漏你隐私

🔗5 本篇只讲专注模式！专注任务中只带轻量记忆，因为！如果你带着完整的全量记忆去做实时视频，它也可以实时，但是会延迟！多出来的那点时间是每一轮吸记忆用掉的。专注模式的做法是换一份更轻的注册表，换来又快又稳的实时感。这个延迟正在研究，等攻克下来有机会再另出一篇。

----- ✦ *🔗* ✦ -----



0. 总纲：还是那一个循环

上份教程第 0 章写过：系统只有一个循环——感知 → 记忆 → 开口/动作。所有模块都是感知源，它们不互相调用，只共享同一份记忆与上下文。这份教程沿用这个循环，因为摄像头只是又一个感知源，跟健康数据、语音、日程完全平级

上份：

[健康/饮食/语音/日程...] → 写入 → [SQLite] → 召回/注入 → [Prompt] → [开口/动作]

这份：

[健康/饮食/语音/日程...] ↴

[耳朵：流式转写] └→ 同一个循环 → [Prompt] → [仲裁器] → [开口/动作]

[眼睛：持续画面理解] ┘

变的只有两处：

1. 注入用哪一份块注册表（第 5 章）
2. 谁有权开口、什么时候开口（第 8 章）

！ **MUST**：单一循环、单一 prompt 装配管线、单一动作执行入口。实时模式复用它们不另起炉灶

！ **NEVER**：为实时模式复制第二条装配管线！这是上一本就写死的红线，到这一本依然是，双实现依旧是最大屎山源



1. 做出来啥样

你打开电脑摄像头开始工作 or 学习。屏幕一侧是机，一侧是你自己的画面。你没跟他说话，也没按任何按钮，但他一直在看着你在写东西、看你停下来发呆、看你换了个姿势、看你拿起了什么或者试图摸鱼。他随时可能开口，说的是此刻真的看到的事。你回一句然后对方接一句。你说到一半他插不进来；你一开口他当场停下。也就是你能插他的嘴但他不能插你的嘴。没错人类拥有绝对话语权！

这个功能跟「语音助手」的根本区别是：助手等你叫他才被激活有反应，陪伴是对方实际上一直在。

这实际上是一个全新的工程问题，原来只有你能让 AI 开口，现在他自己也想开口了。但这会导致两个来源抢同一张嘴，这就是第 8 章要解决的事。人话版你知道有这么回事就行。



2. 为什么能做到，这有上份教程留好的三个口子

这一章是整个教程的核心。很多人以为实时视频要重做一套系统，其实不用，因为上一本那个架构本身就是为这个准备的。

一：分层注入，所以它能快

上一本第 4 章有个块注册表，每一块都标了档位：常驻、触发、条件。装配器只认这张表。

实时模式不是全新的，只是换了一份注册表：只留三块常驻，触发和条件全部关掉于是 prompt 从几万字变成两千字左右。而且这一点比「变短」更重要，核心的两千字在一整场通话里一个字都不变。一模一样，才有资格被模型那边整段缓存，每一轮直接命中、不重新计算，于是可以实现速度快。

但是！注意我说的是才有资格，不是就一定会。缓存不是前缀一样就自动生效，因为多数厂商要在请求里显式声明缓存断点，而且每家的最小长度门槛不同。第 5 章会给你验证的办法。

二：单一循环

上一本第 0 章那条 MUST：所有模块都是感知源，共享同一份记忆和上下文，不互相接线。

摄像头其实就是又一个感知源。它写进来的东西跟健康数据、跟语音标签走的是同一条路。如果你当时听话、没有为不同模式复制第二条装配管线，那么现在你什么都不用改，直接接一个新的感知源进去就完了

口子三：动作标记，他在通话里也能干活

上一本第 5.2 章的动作标记协议 AI 在回复里发暗号，执行器读到才真做，实时模式原样复用。他在通话中说「帮你记一笔」，走的还是同一个执行器、同一张审计表。不用为实时再写一套动作系统。



3. 技术栈：

上份教程的三件套（一台电脑 / Python 3.11 / 一个 LLM API key）继续有效。这次加四样东西：

1. **流式耳朵**。能一边听一边出字，而不是等你说完完整段再上传识别。判断标准：它得支持「持续喂音频进去、持续吐文字出来」。
2. **一双眼睛**。摄像头能持续接收画面，并说出画面里正在发生什么。判断标准：它得支持连续图像输入，而不是一次只能问一张图。
3. **流式嘴巴**。第一个音频字节出来就能开始播，不用等整句合成完。判断标准：它得有流式接口，并且能中途取消。
4. **一条双向的实时通道**。浏览器和你的程序之间要能持续互发数据。WebSocket 就够，不需要更复杂的东西。

以上四样具体用哪一家在第 17 章有一份推荐参考清单，但不是规定死的，具体看自己的喜好和预算。

外加你已经有的：记忆库、prompt 装配器、动作执行器。这三样不用改。

！ MUST：实时服务要跟你原来那个聊天服务分成两个进程跑。实时那条长连接会把原来的服务线程占满，混在一起的后果是聊天页面整个卡死。

！ NEVER：把实时逻辑塞进你现有的 Web 服务里。它们生死要独立：实时那边崩了，你原来的聊天必须照常能用。

3.1 分两个进程之后，数据库如何处理

分进程的设计是对的，但两个进程很可能在读写同一个库文件，尤其当通话里他也要落库、也要执行动作的时候。这一节不写，因为你会遇到偶发的锁死，而且极难复现。但这里可以列出几条规矩帮助你解决：

！ MUST：数据库开 WAL 模式，并设一个合理的忙等超时。这两条是最低成本、收益最大的一对设置

！ MUST：每个进程用自己的连接，连接不跨线程共享。写事务尽量短，把模型调用、网络请求全部挪到事务外面

！ MUST：通话里产生的动作仍然调用你原有的那个统一执行器，不要在实时层另开一条写入路径

！ NEVER：在事务里等任何网络调用。这是 SQLite 锁死最常见的成因，没有之一

验收办法：让实时服务和原聊天服务同时高频写入几分钟，看有没有 `database is locked`。这个测试很粗糙，但它能抓出九成问题。如果出问题，把这个复制给你的机让他去查



4. 如何搭建：分成五步

路线图如下，每一步做完再进下一步，顺序不要换！

1. **先把管子接通。**浏览器和你的程序之间开一条实时通道，音频能上去、声音能下来。这一步没有任何 AI 参与，就是通管子。
2. **装上耳朵和嘴，跑通一问一答。**你说话 → 出字 → 装配 prompt（用新的注册表）→ 调模型 → 合成 → 播放。此时抖动缓冲就要装上，别留到最后搞
3. **把打断做扎实。**你一开口，对方当场停。这才是我理解里类人的精华，而且这一步看起来简单，做扎实很难，我在这就是狂踩坑，它必须在第 4 步之前完成！原因见下面那条 MUST
4. **把眼睛接上，让它一直盯着你。**本地持续采集，上传与观察按节拍自适应（第 7.8 章），每次观察形成一条结构化结果都进你的记忆循环
5. **让他主动开口，但是得知道啥时候该说话，不该说话别说，专注模式就是要专注减少打扰。不然能烦死你。**看到了不等于要说。

！ MUST：第 3 步必须在第 5 步之前做完。因为主动开口会让发言的来源从一个变成两个。仲裁还没做扎实就打开主动发言，你会得到一个抢话、重复、答非所问的东西，而且很难查出是哪一层的毛病。



从这里往后的机读版可以整段复制给你的机，让他/她/它照着实现。前面人话版讲的是为什么，现在讲怎么实现。机读版之后还有一部分人类必读，里面有一些你必须了解的事项，请仔细阅读。

5. 接口与注册表

5.1 记忆系统适配接口你不必非得用我们那套

上一本用的是 SQLite + 块注册表。但如果你的记忆是向量库、知识图谱、是某个编排框架里的一个 state，或者干脆就是几个文本文件，一样能接。因为实时层跟你的记忆之间，只需要这几个函数：

```
# 你的记忆系统只要能提供下面这几个函数，就能接上本书。
# 底下是 SQLite、向量库、知识图谱还是别的，本书一概不管。

# 三个 resident provider 的签名必须完全一致，否则冻结函数没法统一调用。
# 这是初版被审计抓出来的错：接口写了 (store, session)，
# 冻结函数却统一按 (store) 调，照抄下去当场报参数不足。

def load_identity(store, session) -> str:
    # 彼此身份认知。整场通话不变。建议 400-600 字。
    ...

def load_recent_summary(store, session) -> str:
    # 最近若干天的摘要（天数自己在函数里定）。
    # 会话开始时取一次，整场不变。建议 400-600 字。
    ...

def load_task_now(store, session) -> str:
    # 这一场要干的事。会话开始时取一次，整场不变。建议 200-400 字。
    ...

def persist_reply(store, reply_text, turn_id, status) -> None:
    # 落库这一轮的回复。status ∈ {"played", "playback_unknown"}
    # —用户到底听没听到。这是适配层语义，不要求你改原有表结构：
    # 写现有 metadata、扩展字段、审计表或旁路日志都可以。
    ...

def execute_actions(store, reply_text, turn_id) -> list:
    # 可选。解析回复里的动作标记并执行。
    # turn_id 是幂等键：同一个 turn_id 重复提交必须只生效一次。
    # 没有动作系统就返回空表，不影响其余任何功能。
    ...
```

！ **MUST**：这几个函数是本教程与你的记忆系统之间的**唯一接口**。实时层不许直接读你的库，不许直接拼你的 prompt 块

！ **MUST**：这一节所有函数的签名必须跟后面章节里的调用逐字对得上。三个 resident provider 都收 (store, session)；execute_actions 收 (store, reply_text, turn_id) 签名不齐照抄下去会参数不足报错

！ **NEVER**：把实时层跟你的记忆实现耦合。换掉底层存储，不应该需要改实时层的任何一行代码。

5.2 专注模式的块注册表

如果你用的正是我们教程那套，那就沿用第 4 章的注册表格式，只换一份表。新增一个档位 volatile：每轮替换，不累积。如果你们有自己的，就按照自己情况来接

```
REGISTRY_FOCUS = [
    # (name, tier, provider_fn, condition_fn)
    ("identity", "resident", load_identity,      None), # 彼此身份认知
    ("recent",   "resident", load_recent_summary, None), # 最近三天摘要
    ("task_now", "resident", load_task_now,      None), # 这一场要干的事
    ("vision",   "volatile", latest_vision,      None), # 当前画面，每轮替换
]
# 上一本里的 magnet_memory / health_today / work_tarot ... 一律不进场。
```

5.3 resident 必须在会话开始时冻结

这是最容易写错的地方之一。如果你每一轮都重新调一次 load_recent_summary，那「整场不变」就是纯吹牛逼，provider 里只要有一处读了「现在」，前缀就会漂移，缓存全程不命中，而且没有任何报错，非常之鸡贼

正确做法：会话开始时生成一次快照并冻结，整场不再调用 provider

```
# — 会话开始时冻结一次，整场不再重算 —————
def freeze_resident(store, session):
    blocks = []
    for name, tier, provider, _ in REGISTRY_FOCUS:
        if tier != "resident":
            continue
        text = provider(store, session) # 签名统一，见 5.1
        if not text:
            continue # 空块整块消失，沿用上一本
        blocks.append(f"[{name}]\n{text}")
    session.resident = "\n\n".join(blocks) # 冻结的缓存前缀
    session.resident_hash = sha256(session.resident)
    return session.resident

# — 每轮只装配变化的部分 —————
def assemble_focus(session, user_text):
    volatile = []
    for name, tier, provider, _ in REGISTRY_FOCUS:
        if tier != "volatile":
            continue
        text = provider(session) # vision: 取最新一条，替换语义
        if not text:
            continue
        volatile.append(f"[{name}]\n{text}")
    return session.resident, "\n\n".join(volatile), user_text
#      ↑ 冻结前缀      ↑ 每轮替换      ↑ 当轮输入
```

注意这里没有用「先过滤掉空块、再按下标把属性对回去」那种写法。一旦中间某个块返回空，下标就会错位，volatile 甚至可能被误算进 resident 前缀里。上面这个写法在同一个循环里判档位，不存在错位的可能，写出来给你避坑。

5.4 换版规则

resident 不是永远不变，是“一场通话里不变”。换版只在明确的事件下发生：user 换了这一场要干的事、跨了自然日、或者 user 手动重置。

！ MUST：换版 = 结束当前会话、开一场新的。绝对不许在会话中途悄悄替换 resident！那会让缓存失效、上下文断裂，而 user 完全不知道发生了什么

5.5 怎么确认缓存真的生效了

！ MUST：缓存不是前缀一样就自动命中。多数厂商要你在请求里显式声明缓存断点，各家字段名不同，而且各家的最小长度门槛不同。开工前先读一遍你那家的缓存文档

！ MUST：验证要看响应里的缓存用量字段，命中了多少 token、写入了多少 token。本地哈希相同只是必要条件，不是充分条件。

！ NEVER：假设缓存已经生效就继续往下做。第一次跑通之后，去看一眼响应的用量字段确认命中率，再决定要不要继续优化。不看这一眼，你可能整套做完了却一秒都没省到。

重要备注：resident 三块加起来建议落在 1200-1500 字。不需要为节省 token 而=压缩的很短很短。因为写太短反而可能更慢！想不到吧哈哈哈！！多数模型的缓存有最小长度门槛，常见 1000 token 起，低于门槛就缓存不上了。是不是很反直觉？

6 耳朵：一边听一边出字

采集：16kHz / 单声道 / PCM16, 20ms 一帧，持续上行。

收尾判定（判断“他说完了”）：

```
silence_ms = 300          # 越短越快，越容易抢话
误收尾兜底：生成开始后 1500ms 内用户重新开口
    → 代际 +1，取消当前生成
    → 计入“误收尾率”指标，但不计入故障告警
```

！ **MUST**：转写要区分中间结果与最终结果。中间结果只用来显示字幕，绝不进入 prompt，也绝不落库。

！ **MUST**：采集必须开回声消除、开噪声抑制、**关自动增益**。自动增益会把回声一起放大。

！ **NEVER**：把误收尾当故障报警。误收尾是这套设计的正常行为，把它当故障，告警总线会被刷屏，真事故就被淹没了。

回声自激是这类项目最常翻的车，**我必须单独说一句**，这个很重要而且一般很容易被遗漏。用外放的话，它的声音会被你的麦克风收回去，被当成你在说话，于是它自己打断自己，来回抽风。除了上面那条 MUST，还要加一道**双讲保护**！**播放期间提高触发门限并要求持续时长**！如果每分钟自打断两次以上，自动降为半双工并提示 user 戴耳机。

----- ✦ * 79 * ✦ -----

7. 眼睛：一直在看，不能够被削减的部分

两只眼，一只在浏览器里，一只在云上。这是分工，不是备份。本地那只跑在浏览器，帧率可以到几十帧，判断**粗**（在不在、动没动、姿态大概/状态/在干啥/什么表情），画面不出你的电脑。它给的是反应速度和隐私。云上那只是真正看得懂的那只，虽然帧率受接口限制（目前主流大约每秒一帧）。它给的是理解力 判断**细**。

！ **MUST**：本地采集与本地眼**持续运行、不间断**；云端上传与认知观察按第 7.8 章的三层节拍自适应，并且必须设置最长观察间隔。它是一直在看，不是被叫醒才看「一直在看」由本地层保证，「看得懂」由自适应的上传与观察保证。

！ **NEVER**：把云眼做成“出事了才看一眼”。很多人本能地会拿本地眼当守门员、发现异常才叫醒云眼——那样确实省钱，但省出来的是一个只会念你预设规则的报警器，不是陪伴。

！ **NEVER**：以省钱为由降低本地采集与本地眼的频率。这一层不产生任何 API 费用，降它一分钱省不到，却会真的让它看不见。至于上传帧率和让模型「想一想」的节拍。那两层花钱是主要烧钱的地方，所以如果要抠一点可以也应该按证据自适应，见第 7.8 章，账见第 16 章。

唯一该保留的限制是失控熔断：连续失败、并发失控、异常循环。那是防代码写错了无限调用，不是省钱。这两件事必须分开，别混为一谈。

每次观察产出一条结构化结果，进上一份教程说过的那个循环：

```
{
  "observed_at": "<时间戳>",
  "ttl_ms": 4000,           # 超时即作废，过期的画面不许用现在时复述
  "scene": "<画面里正在发生什么>",
  "subject_state": "<用户当前的状态>",
  "changed_from_last": true,
  "salience": 0.0,         # 值不值得开口，0..1，由感知层给"建议分"
  "uncertain": false       # 看不清就置 true，不许猜
}
```

！ **MUST**：原始帧只在内存里待着。绝不落盘、绝不进日志、绝不进备份。只有上面这条结构化观察可以持久化。

！ **MUST**：salience 是建议分，不是决定。感知层给分，开不开口由生成层定，因为生成层知道上下文，感知层不知道。

！ **NEVER**：让感知层直接触发说话。那等于把「什么时候该开口」写死成规则，你会得到一个规则机器人，而不是一个在看着你的伙伴。

7.6 送帧 ≠ 它在看！

这一条是实现层的大坑，纸面上完全看不出来。有的实时接口收到画面帧，只是把它放进上下文里待着，并不会自己起一轮推理。你以为我一直在送帧，所以它一直在看，实际上它一帧都没「thinking」过，直到你说话它才醒。

！ **MUST**：查清你选的接口：只送帧会不会产生观察？如果不会，你要自己按节拍发一个观察请求（一句固定的提示 + 最新一帧），才会有结构化观察出来。这个观察请求就是你的「看一眼」节拍器。

！ **NEVER**：假设「送帧 = 它在看」。送帧和产生观察是两件事，这一条不验证，你会做出一个看起来一直开着摄像头、实际上什么都没看的東西。

7.7 长会话撑不了半小时

这一条我们是在反侧测试中抓出来的，而且它会让人在演示的第三分钟当场翻车。

实时接口的单次会话是有硬性时长上限的，而且「带视频的会话」通常比「纯音频的会话」短得多。举个真实的例子：某家官方文档明确写着，音频加视频的会话限制在两分钟。

也就是说，你想要的半小时通话，在实现上不是一条连着的长连接，而是：

多段会话 + 恢复接续：

会话 1 (到期前主动重建) → 会话 2 → 会话 3 → ...
├─ resident 快照原样搬过去 (它本来就是冻结的，正好)
├─ 滚动对话只带最近 N 轮的摘要，不带全文
└─ 处理服务端的主动告别信号 (很多接口关闭前会先通知)

另一条上限来自上下文窗口 (常见 32k / 128k token)，到了也要压缩，机制跟上面一样。

！ **MUST**：查清你选的接口的单次会话时长上限与上下文窗口上限，并按「多段会话 + 恢复接续」来设计。

！ **NEVER**：把「一条连接撑半小时」写进你的设计假设。

这里有个意外的好消息：因为 resident 是会话开始时冻结的快照 (第 5.3 章写了)，重建会话的时候原样搬过去就行，前缀仍然具备缓存复用的资格。注意是「具备资格」不是「一定续上」缓存能不能跨连接、跨会话继续有效，取决于厂商、缓存有效期和它的作用域，重建之后请照 5.5 章再看一眼用量字段。但如果你当初图省事让 resident 每轮现算，这会儿重建会话就会变成一场究极噩梦！这两章是配套的请不要省略。

7.8 奇迹发生在此的三个节点：采集、上传、思考

这个结构我们试了很多次才拆干净，这是最核心的难点！频率其实有三层，每层的成本来源不同，所以每层的规则不同：

CAPTURE_FPS	本地摄像头采集 + 本地眼推理 # 不产生任何 API 费用，只吃你自己的设备。 # 规则：拉满（以设备发热和掉帧为限）。
UPLOAD_FPS	实际上传给云模型的帧率 # 花钱：输入 token + 带宽 + 累积上下文。 # 规则：可调。按画面变化率自适应， # 变化大收紧、没变化放缓，永远只传最新帧。
OBSERVE_MIN_S	= 1.0 # 认知节拍下限：最快多久让模型"想一次"
OBSERVE_MAX_S	= 10.0 # 认知节拍上限：画面再没变化，也至少想一次 # 最花钱的一层：每次"想"都有几十上百个输出 token。 # 规则：按变化率自适应，但必须有上限保底。
联动：本地眼察觉明显变化 / 用户刚说完话 / 刚开始新任务 → UPLOAD 收紧 + 立刻触发一次观察，不等节拍	

奇迹在这里发生！

三层分开之后，每一层的问题都变简单了。「一直看着你」由采集层保证,摄像头真的一直开着，本地眼真的一直在跑，任何变化都会在毫秒级被察觉。「看得懂」由上传层和认知层保证,变化一出现就收紧、就真的去思考。「不破产」由自适应保证,你安静工作的那些分钟里，上传和推理自动放缓，钱实际上花在有信息量的时刻

！ **MUST**：三个节拍必须是三个独立参数。采集层绝对不为省钱降；上传层与认知层可以按证据自适应，依据是接口上限、实际账单、设备负载、画面变化率与实测体感，不是「感觉贵」。

！ **MUST**：认知节拍必须有上限保底。画面再怎么没变化，也要定期真的想一次。没有这条，它就退化成「本地眼当守门员、有事才叫醒模型」的报警器，那正是第 7 章一开始就禁掉的东西。

！ **MUST**：无论节拍怎么调，上行永远只保留最新帧。跟不上就丢旧的，绝不排队堆积。

！ **NEVER**：把三层绑成同一个数。绑在一起你只有两个选择：要么贵得离谱，要么它其实没在看。

这个三层区分同时解开了三件事：我们最终效果要的「一直看着我」（采集层保证）、工程要的「反应快」（变化驱动的联动保证）、账单要的「防止你倾家荡产」（自适应保证）。三者的冲突只存在于把它们混为一谈的时候。这个分开的设计就规避了这点。



8.发言权仲裁：一张嘴两个人抢怎么办

这是本教程唯一真正新增的架构件。上面所有部件都可以换实现，这一章的规则不能改。

idle —用户开口→ listening —转写终态→ generating → speaking → idle

idle —视觉触发→ generating → speaking → idle

speaking —用户开口→ interrupted → listening

任意态 —致命错误→ closing

代际位: epoch (只增不减)

优先队列: 0 用户打断 / 1 会话关闭 / 2 用户语音 / 3 发言流事件 / 4 视觉观察 / 9 遥测

一次发言 = 一个"发言流"。它有且只有两个对外事件:

turn_speaking 真的开始出声了

turn_done 彻底结束了 (说完 / 沉默 / 异常 / 被打断, 都发这个)

！ **MUST**: 同一时刻只有一个发言流处于开放状态。

！ **MUST**: 用户永远优先。正在说话时收到用户语音，立即停止播放——不等当前句子说完，不等云端确认取消，当场静音。

！ **MUST**: 每次发言携带一个单调递增的代际编号。代际变了之后才到达的旧结果，一律丢弃，不许播出。

！ **MUST**: 视觉触发只在 idle 态被接受。在听、在想、在说的时候来的视觉观察，排队或丢弃，不许抢占。

！ **MUST**: 生成层有权返回空 = 选择沉默。沉默不消耗发言权，也不改变最终状态。

！ **NEVER**: 让仲裁器等待任何模型调用。它只做三件事：校验、排序、改状态。所有耗时操作丢出去跑，完成后把事件投回队列。

8.1 两个最容易写错的地方

这两条写在这里给你避坑。我们自己都写错过，你大概率也会。

错法一：沉默把状态机卡死。

你写了「收到沉默事件就回到 IDLE」，但生成函数在没有句子的时候什么都不发——于是那个沉默事件永远不会到来，状态永久停在 GENERATING，之后所有视觉触发全部被挡在门外。整个东西看起来还活着，但它再也不会主动开口了。

！ MUST：一次发言必须有且只有一个终态事件。说了几句、一句没说、还是中途异常，都要在 finally 里发 TURN_DONE。

错法二：多句并发，播出乱序

你每取到一句就并发丢出去合成，看起来更快 but 实际上句子会乱序播出，而且任意一句先报「我说完了」就把状态提前放回 IDLE，直接违反上面那条「同一时刻只有一个发言流」。这个 bug 在你自己测的时候很难复现，因为短句合成快，顺序碰巧是对的。

！ MUST：同一次发言里的多个句子必须串行 await，不许并发。只有整个发言流结束才发 TURN_DONE，单句完成不发。

！ NEVER：让单句的完成事件把状态放回 IDLE。能把状态放回 IDLE 的，只有 TURN_DONE 一个。

错法三：两个发言流同时开起来。

收到转写终态就直接开一个发言流，没有检查当前状态，如果同一轮到了两个终态（重连补发、供应商重试、你自己的去重漏了），就会同时开出两个流，两张嘴一起说话。

！ MUST：取得发言权必须是一个原子操作，用户语音和视觉触发都走同一个入口，谁也不许绕过去直接开流。具体写法见第 10.1 章。

第三条规则（代际编号）最不直观，但最要命。它一秒前决定要说的那句话，一秒后可能已经不合时宜了，因为你已经在说别的了。有了代际编号还有个隐藏好处：打断不需要等任何东西完成，你一开口编号加一，正在生成的、正在合成的、正在排队播的全部作废，不用挨个取消。这是「反应快」和「反应正确」能同时成立的关键。

不做这几条，症状分别是：他打断你、他喋喋不休、他答非所问、或者某一刻之后再也不主动说话了。这些事情都非常下头，这几条已经把下头问题解决掉了。



9. 说话与播放：主动留那 180 毫秒的延迟是故意的

合成侧要流式：第一句一出来就送去合成，不等它说完整段。输出格式优先选无损分块（PCM 这类），有损分块格式在块与块之间会有能听见的缝。

播放侧要有抖动缓冲！

因为网络包到达的时间永远不均匀！不攒直接播，声音会一顿一顿。

```
const TARGET_BUFFER_MS = 180; // 目标水位，范围 120-320 自适应
let cur = { epoch: -1, turnId: -1 }; // 当前唯一接受的流
let streamEnded = false;
let finishedAckSent = false; // 播放完成回执只发一次
let nextPlayTime = 0;
let scheduled = [];

// — 每个新流的第一件事：换流并重置 —————
function onStreamStart(epoch, turnId) {
  cur = { epoch, turnId };
  streamEnded = false; // ← 漏掉这行，上一轮留下的 true
  finishedAckSent = false; // 会让下一轮第一块播完就提前收场
  nextPlayTime = 0;
  scheduled.forEach(s => { s.onended = null; try { s.stop(); } catch (e) {} });
  scheduled = [];
  gainNode.gain.cancelScheduledValues(audioCtx.currentTime);
  gainNode.gain.setValueAtTime(1, audioCtx.currentTime);
}

function isCurrent(epoch, turnId) {
  return epoch === cur.epoch && turnId === cur.turnId;
}

function onChunk(pcm, epoch, turnId) {
  if (!isCurrent(epoch, turnId)) return; // 旧流迟到的音频，丢
  gainNode.gain.cancelScheduledValues(audioCtx.currentTime);
  gainNode.gain.setValueAtTime(1, audioCtx.currentTime); // 欠载淡出后恢复

  const buf = toAudioBuffer(pcm);
  const src = audioCtx.createBufferSource();
  src.buffer = buf; src.connect(gainNode);

  const now = audioCtx.currentTime;
  if (nextPlayTime < now + TARGET_BUFFER_MS / 1000)
    nextPlayTime = now + TARGET_BUFFER_MS / 1000; // 首块：先攒水再播
  src.start(nextPlayTime);
  nextPlayTime += buf.duration;

  src.onended = () => {
    scheduled = scheduled.filter(x => x !== src);
    maybeFinished(epoch, turnId); // 出口一：最后一块后播完
  };
  scheduled.push(src);
}

function onStreamEnded(epoch, turnId) { // 服务端：这一轮音频发完了
  if (!isCurrent(epoch, turnId)) return;
  streamEnded = true;
  maybeFinished(epoch, turnId); // 出口二：通知晚于最后一块播完
}
```

```

}

function maybeFinished(epoch, turnId) {      // 两个出口共用的唯一判定
  if (finishedAckSent) return;               // 幂等：重传的流结束通知不重复回执
  if (streamEnded && scheduled.length === 0 && isCurrent(epoch, turnId)) {
    finishedAckSent = true;
    send('playback.finished', { epoch, turnId });
  }
}

function onStreamAbort(epoch, turnId) {      // 服务端异常：立刻停 + 回执
  if (!isCurrent(epoch, turnId)) return;
  stopAllScheduled();
  send('playback.stopped', { epoch, turnId, reason: 'aborted' });
}

function onEpochChange(newEpoch) {        // 用户打断：当场停 + 回执
  const stopped = { ...cur };                // ← 先记下被停掉的是哪一个流
  stopAllScheduled();
  cur = { epoch: newEpoch, turnId: -1 };
  if (stopped.turnId !== -1) {
    send('playback.stopped', {
      epoch: stopped.epoch,                  // 回执带的是【旧流】的标识，
      turnId: stopped.turnId,               // 因为服务端等的就是旧流的回执。
      newEpoch,                             // 新代际单独一个字段带回去。
      reason: 'interrupted',
    });
  }
}

function onUnderrun() {                      // 水池见底
  metrics.underrun++;
  gainNode.gain.linearRampToValueAtTime(0, audioCtx.currentTime + 0.02);
  // 20ms 淡出，不做静音跳变。恢复由下一次 onChunk 负责。
}

function stopAllScheduled() {
  scheduled.forEach(s => { s.onended = null; try { s.stop(); } catch (e) {} });
  scheduled = []; nextPlayTime = 0;
  streamEnded = false; finishedAckSent = false;
  gainNode.gain.cancelScheduledValues(audioCtx.currentTime);
  gainNode.gain.setValueAtTime(1, audioCtx.currentTime);
}

function teardown() {                       // 页面关闭 / 挂断
  stopAllScheduled();
  gainNode.disconnect();
  audioCtx.close();                         // 不关它，音频线程会一直挂着
}

```

！ **MUST**：所有播放相关消息都必须携带 `(epoch, turn_id)` 二元标识：音频块、流结束、流中止、播放完成、播放停止，一个都不例外。只带 `epoch` 不够——同一代际里可以连续发生多次视觉发言，上一轮迟到的数据会被下一轮误收。

！ **MUST**：打断回执带的必须是【被停掉的那个旧流】的 `(epoch, turn_id)`，不是新代际。服务端等的是旧流的停止确认；回一个新 `epoch` 过去，它永远匹配不上，异常路径那个短超时每次都会走满。新代际另开一个字段带回去。这是第五轮审计抓出来的——我们自己刚立的规矩，自己在这个分支里破了。

！ **MUST**：播放完成回执必须幂等。流结束通知可能被重传，一个 `finishedAckSent` 标志位就够——新流开始时跟着一起重置。

！ **MUST**：每个新流开始时必须重置客户端状态：换 `(epoch, turnId)`、`streamEnded` 归 `false`、清空排程、恢复音量。漏掉 `streamEnded` 那一项，上一轮留下的 `true` 会让下一轮的第一块音频一播完就提前宣布整轮结束。

！ **MUST**：播放必须有抖动缓冲，且欠载时做淡出而不是静音跳变；欠载淡出之后，下一块音频到达时必须把音量恢复——只写淡出不写恢复，一次欠载之后整通电话再也没有声音，而且不报任何错。

！ **MUST**：「流结束」和「最后一块播完」是两个独立事件，两支都要能触发播放完成的确认。上面的 `maybeFinished` 就是两支共用的唯一判定——只挂一边，另一条到达顺序就会让服务端白等到超时。

！ **MUST**：已播完的音频节点从表里移除；被打断或中止时连同 `onended` 一起摘掉；挂断或页面关闭时释放音频上下文与所有节点。浏览器不会替你收拾。

！ **NEVER**：用「播分块 MP3」那类做法。块边界的缝会被听成卡顿，而你会以为是网络问题，查错方向。

这 180 毫秒是整套设计里唯一主动变慢的地方。为什么非得这样，因为人对「卡」的体感忍耐度绝对远低于对「慢」的忍耐度，半秒你不一定察觉，断一下你立刻就烦。反正我忍不了

9.1 消息协议：一张表，防漏字段

我们落地过程中好几次发现同一类问题：某条消息漏带了标识字段。原因不是粗心，是消息的定义散落在两段代码和一张出口表里，因为散着定义就会散着漏。

所以把浏览器与服务端之间的消息集中成一张表。加字段、加消息，都从这里改：

方向	消息	必带字段	含义
S→C	audio.stream_start	epoch, turn_id	新发言流开始 客户端据此重置状态
S→C	audio.chunk	epoch, turn_id, pcm	一块音频
S→C	audio.stream_ended	epoch, turn_id	这一轮音频发完了
S→C	audio.stream_abort	epoch, turn_id	异常：立刻停播
S→C	ctrl.stop_playback	new_epoch	用户打断，全部停
S→C	turn.error	epoch, turn_id, reason	告诉用户这轮失败了
C→S	playback.finished	epoch, turn_id	真的播完了（幂等，只发一次）
C→S	playback.stopped	epoch, turn_id, reason [+ new_epoch]	已停播 ← 打断时另带新代际
C→S	audio.frame	seq, epoch	上行麦克风帧
C→S	vision.frame	seq, ts	上行画面帧（独立通道）

铁律三条：

1. 凡是跟某一次发言有关的消息，epoch 和 turn_id 一个都不能少。
2. 回执带的永远是【被回应的那个流】的标识，不是当前最新的。
打断时尤其容易搞反——服务端等的是旧流的确认。
3. 新增字段先改这张表，再改两边代码。

！ MUST：这张表是消息定义的唯一真相源。两边代码都从它推导，不许某一边自己加字段——第五轮审计抓到的打断回执缺 turn_id，就是因为那段代码没对着表写。



10. 骨架代码：一轮的必经路径

把上面各章串起来。三个要点：仲裁循环里没有一处 await 模型调用；发言权是原子取得的；终态以浏览器真的播完为准。

10.1 发言权是抢来的，而且要按来源限定状态

如果收到转写终态就直接开一个发言流，没有检查当前状态。如果同一轮到了两个终态（重连补发、供应商重试、或者你自己的去重没做好），就会同时开出两个发言流，两张嘴一起说话。

我们踩过的一个坑：它同时允许 IDLE 和 LISTENING 两种状态。于是 user 正在说话（LISTENING）的时候，一条高分视觉观察照样能抢到发言权,这里直接违反「user 永远优先」的铁律

正确做法是：取得发言权时按来源指定它允许的状态。

```
def try_acquire_floor(session, epoch, expected_state):
    # 取得发言权。成功返回 turn_id, 失败返回 None。
    # 这是整套仲裁里唯一允许开启发言流的入口。
    if session.state != expected_state:
        return None          # 状态不对，这个来源现在没资格说话
    if session.active_turn is not None:
        return None          # 已经有发言流开着
    if session.epoch != epoch:
        return None          # 代际过期
    session.turn_seq += 1
    session.active_turn = session.turn_seq
    session.state = GENERATING
    return session.active_turn

# 用户最终转写：只允许在 LISTENING 时开口
tid = try_acquire_floor(session, epoch, expected_state=LISTENING)

# 视觉触发：只允许在 IDLE 时开口
tid = try_acquire_floor(session, epoch, expected_state=IDLE)
```

！ MUST：开启发言流的唯一入口是这个原子操作，并且必须按来源指定 expected_state：用户语音只在 LISTENING、视觉只在 IDLE。不指定的话，视觉就会在你说话说到一半的时候插进来。

！ MUST：同一轮只接受一次最终转写。用 turn_id 去重，重连补发或供应商重试送来的重复终态一律丢弃。

10.2 仲裁循环

```
async def orchestrate(session):
    session.epoch, session.state = 0, IDLE
    session.active_turn, session.turn_seq = None, 0

    async for ev in session.queue:
        # 优先队列，见第 8 章
        if ev.epoch is not None and ev.epoch != session.epoch:
            metrics.late_discard += 1
            continue
            # 代际过期，丢弃

        if ev.type == USER_SPEECH_START:
            session.epoch += 1
            session.active_turn = None
            session.stop_playback(session.epoch)
            session.cancel_inflight()
            session.state = LISTENING
            # 用户永远优先
            # 旧发言流作废
            # 当场停，不等任何确认
            # 取消失败也不阻塞

        elif ev.type == ASR_FINAL and session.state == LISTENING:
            if session.seen_final(ev.turn_key):
                continue
            # 同一轮只收一次
            tid = try_acquire_floor(session, ev.epoch, LISTENING)
            if tid: spawn(speak_turn(session, ev.text, USER_SPEECH, ev.epoch, tid))

        elif ev.type == OBSERVATION:
            if ev.is_expired(): continue
            session.ctx.vision = ev
            if ev.salience >= THRESHOLD:
                # 只允许在 IDLE 抢—用户在说话时视觉一律抢不到
                tid = try_acquire_floor(session, session.epoch, IDLE)
                if tid: spawn(speak_turn(session, None, VISION, session.epoch, tid))
            # 第 7 章：过期观察直接丢
            # 第 5 章：替换，不累积

        elif ev.type == TURN_SPEAKING:
            if ev.turn_id != session.active_turn:
                continue
            session.state = SPEAKING
            # 旧流迟到的事件，不污染新轮

        elif ev.type == TURN_DONE:
            if ev.turn_id != session.active_turn:
                continue
            session.active_turn = None
            session.state = IDLE
            # 唯一能放回 IDLE 的事件
            # 旧发言流的终态，不改新状态
```

10.3 发言流：每个出口都要善终

初版的错是终态发在「音频发完」；第二版改成等「浏览器播完」，但异常路径没跟上，模型或合成在已经播了几句之后报错，代码跳进异常分支，立刻发终态，可浏览器里还有音频在响。状态回了 IDLE，声音还没停，视觉触发又能进来了。

教训是：不能只把正常路径修对。一次发言有七个出口（见 10.6），每一个都要走到干净的终点。

```
async def speak_turn(session, user_text, trigger, epoch, turn_id):
    # 一次发言 = 一个发言流。句子在流内串行，绝不并发。
    # 无论从哪个出口离开，finally 必发 TURN_DONE。
    spoke, sentences = False, []
    markers = MarkerStream()  # 跨块标记解析器，见 10.5
    try:
        prefix, volatile, text = assemble_focus(session, user_text)
        async for sentence in brain.stream(prefix, volatile, text):
            if session.epoch != epoch: return  # 出口⑤ 生成期间被打断
            sentences.append(sentence)
            speech = markers.feed(sentence)  # 剥标记：半截标记留在解析器里
            if not speech: continue
            if not spoke:
                session.send_client(AUDIO_STREAM_START, epoch, turn_id)
                post(session.queue, TURN_SPEAKING, epoch=epoch, turn_id=turn_id)
                spoke = True
            await mouth.speak(speech, epoch, turn_id)  # await: 串行，保序

        session.mark_turn_completed(turn_id)  # ← 只有正常走完生成循环才置位
        # 异常、取消、打断都到不了这一行，所以动作门的第③条才有意义。
        # 漏了它，门永远返回 False，动作一个都执行不了。

        if spoke:
            tail = markers.flush()  # 解析器里最后的残余文本
            if tail: await mouth.speak(tail, epoch, turn_id)

        if not spoke:
            return  # 出口② 沉默：不等播放

        session.send_client(AUDIO_STREAM_ENDED, epoch, turn_id)
        played = await session.await_playback_finished(
            epoch, turn_id, timeout=8.0)

        reply = "".join(sentences)  # 先合成一份完整回复
        if session.epoch != epoch:
            return  # 出口⑤ 播放期间被打断
        if not played:
            session.persist(reply, turn_id, status="playback_unknown")
            return  # 出口⑥ 确认超时/连接断开
            # 落库存疑，绝不执行动作
        session.persist(reply, turn_id, status="played")
        # status 是【适配层语义】，不要求你改动原有数据库结构：
        # 写进现有的 metadata / 扩展字段 / 动作审计表 / 旁路日志都行，
        # 只要事后能分辨"这条回复用户到底听没听到"。
        if may_execute_actions(session, epoch, turn_id, played):  # 见 10.4
            execute_actions(session.store, reply, turn_id)
        # 出口① 正常善终

    except Exception as e:  # 出口③④ 生成/合成异常
```

```

log.warning("turn %s failed: %s", turn_id, e)
session.send_client(TURN_ERROR, epoch, turn_id)
# ↑ 真的告诉用户出错了。只写日志不叫"出声"—
# 用户那边只会是一段莫名其妙的沉默。
if spoke:                                # 已经播出去一部分了
    session.send_client(AUDIO_STREAM_ABORT, epoch, turn_id)
    await session.await_playback_stopped(epoch, turn_id, timeout=2.0)
    # 先把已排程的音频停干净、拿到回执（或短超时），
    # 才允许 finally 把状态放回 IDLE。
finally:
    post(session.queue, TURN_DONE, epoch=epoch, turn_id=turn_id)

```

！ **MUST**：异常路径必须先安顿好正在播的音频，才能发终态：发流中止、等浏览器的停止回执（短超时兜底），然后才轮到 TURN_DONE。跳过这一步，状态回了 IDLE 而声音还在响，视觉触发就会踩着上一句的尾巴开口。

！ **MUST**：异常必须发一个明确的客户端错误事件让用户知道这轮失败了。写日志不算通知，日志是给你看的，不是给听的。

！ **MUST**：等播放确认要设超时，但超时不等于播完（见 10.4）。宁可这一轮的动作作废，也不要永远卡在 SPEAKING。

！ **NEVER**：把「音频发送完成」当成一次发言的结束。发完不等于播完，中间还隔着一整个抖动缓冲。

10.4 动作门：四个条件，缺一不可

我们测试的前三版里「动作能不能执行」的条件散在四五处 MUST 里，结果每一轮审计都能找到一条漏掉的分支：第二轮漏了「整个没接」，第三轮漏了「播放期间被打断」，第四轮漏了「确认超时」。

散着写就会散着漏。这一版把它收成一个门，只有一个入口：

```

def may_execute_actions(session, epoch, turn_id, played):
    return (session.epoch == epoch          # ① 全程未被打断（含播放期间）
            and played is True              # ② 浏览器确认真的播完了
            and session.turn_completed(turn_id) # ③ 生成完整走完，不是异常退出
            and not session.executed(turn_id)) # ④ 这一轮还没提交过（幂等）

# 四个条件同时成立，动作才提交。高风险动作（花钱、发消息、删东西）
# 在这四条之上，还要等用户明确确认。

```

！ **MUST**：播放确认超时后绝不执行动作。浏览器可能已经崩了、断线了。User 根本没听见他说「我帮你记一笔」，钱却已经划走了。回复落库并标记 playback_unknown，动作作废，等 user 回来自己看。

！ **MUST**：动作只从这一个门进。任何地方想执行动作，都调用这个判据，不许自己另开条件。

！ NEVER：把动作提交安排在等待播放确认之前。测试有抓过一次：user 在播放期间打断，动作已经执行了，我们自己写的规矩，被我们自己的代码顺序违反。

10.5 动作标记会被流式输出拆成两半

上一本第 5.2 章用文字标记加正则解析动作，那个做法在上一个教程里是对的因为那里是一次性拿到完整回复再解析，标记永远是完整的。

流式把前提改了：模型逐块吐字，一个标记完全可能被拆在两块里送出来。前半块看起来是普通文字，直接送去合成，他就会把半个暗号念出来；后半块单独解析又会失败。

两条路，按优先级：

1. **最佳：结构化通道。**如果你的模型接口支持工具调用（function calling），动作走结构化通道，朗读文本里根本不出现标记，拆分问题从根上消失。新项目直接选这条。
2. **兜底：跨块解析器。**必须沿用文字标记的话（比如要跟上一本的执行器兼容），解析器必须跨块保留「可能是半个标记」的尾巴，等下一块拼上再判：

```
class MarkerStream:
    # "[WAL" + "LET:out:35]" 这样被拆开的标记，靠尾巴缓存拼回来。
    def __init__(self):
        self.tail = ""
    def feed(self, chunk) -> str:
        buf = self.tail + chunk
        speech, self.tail = split_complete_markers(buf)
        # split_complete_markers:
        #   完整标记 → 剥出执行；确认无标记的文本 → 返回去朗读；
        #   疑似半截标记的尾部 → 留在 tail，这一轮不朗读它
        return speech
    def flush(self) -> str:
        t, self.tail = self.tail, ""
        if not t:
            return ""
        if is_plain_text(t):           # ① 确认是普通文本 → 朗读
            return t
        speech, leftover = split_complete_markers(t + SENTINEL)
        if leftover:                  # ③ 疑似未闭合的半截标记：
            log.warning("unterminated marker tail: %r", leftover)
            self.broken = True        #   不朗读、记解析错误、
            return speech              #   并且本轮不执行任何动作
        return speech                 # ② 完整标记已剥离，剩下的照读
```

！ MUST：朗读文本里绝不允许出现标记或半个标记。流结束时残余分三种处理：确认是普通文本就朗读；完整标记剥离执行；疑似未闭合的半截标记则不朗读、记一条解析错误、并且本轮不执行任何动作。含糊不清的暗号宁可整轮作废，也不能猜着执行。

！ MUST：测试办法：构造一个恰好在标记中间断开的流，断言合成侧收到的文本干净、完整标记照常执行；再构造一个流到一半就断的，断言那一轮动作全部作废。

10.6 一次发言的七个出口

这张全景图是四轮失败测试换来的。每一轮都有一个出口没走干净：第一轮是②，第二轮是①，第三轮是⑤和②的超时变体，第四轮是③④⑥。一次列全，照着验：

出口	客户端看到什么	动作执行？	终点
① 正常播完	流结束 → 回"播完"回执	过门才执行	TURN_DONE
② 沉默	什么都没收到	否	TURN_DONE（不等播放）
③ 生成中途异常	错误提示 + 流中止 → 回停止	否	等停止回执 → TURN_DONE
④ 合成中途异常	同 ③	否	同 ③
⑤ 被用户打断	停播指令 → 回"已停"回执	否	epoch+1，旧流全作废
⑥ 播放确认超时	（无回执—可能已崩/断线）	否	落库标 unknown → TURN_DONE
⑦ 连接断开	—	否	按 ⑥ 处理，会话转入重连

七条路，全部终结于 TURN_DONE；动作只可能在 ① 且过门时发生。

！ MUST：验收时七条出口逐条演练，每条都要确认：状态回到了 IDLE、客户端没有残留音频、动作该执行的执行了、不该执行的一个都没执行。只测正常路径的系统，出的全是看起来能跑的鸡贼事故。

10.7 一轮的必经步骤

！ MUST：

- ①收到触发
- ②按来源原子取得发言权
- ③装配（前缀取冻结快照，volatile 每轮替换）
- ④调模型逐句取 ⑤经跨块解析器剥标记后逐句串行送合成
- ⑥沉默则直接结束，否则等浏览器确认播完
- ⑦过动作门才落库提交动作
- ⑧无论从哪个出口离开都发出终态事件。

八步缺一不可，顺序不可换。

！ NEVER：在 speak_turn 里省掉 finally。这是全教程里看起来最省事但是暴雷的时候会让你哭得最大声的一处偷懒：)

11 你必须自己带来的东西，本规格无法提供的：

1 依旧是那三块常驻内容的正文

你是谁、他是谁、你们是什么关系。最近记忆摘要怎么写入。这一场的专注是要干什么。什么时候来提醒你。这是你们人格，不是本教程能替你写的。

2 他觉得值得开口的尺度

salience 阈值调多少，是这套东西有没有分寸的关键。偏低会让他话痨变得很烦，偏高会像块木头。从中位往两边试，只能按照你自己的需求自己试。

3 一台有摄像头而且你真的会用的机器

持续视觉观测会发热、耗电、掉帧，这件事必须在你真正会用的那台机器（比如笔记本电脑）上跑满十分钟你感觉一下。

12. 零基础起步：前提与选路

前八章是规格，这一章起是可执行路径。

！ MUST：环境四件套：

①上一本那套已经跑通的记忆系统

②一个支持流式的语音识别接口

③一个支持连续图像输入的多模态接口

④一个支持流式的语音合成接口。②③常常是同一家，可以合并。

人类要做的事情：给你的机发这个（直接照抄给他）

注意第一段是「阶段 0：先不要写代码」，这个务必先执行，别嫌麻烦！

如果不先侦察就直接生成，AI 会按想象中的项目结构写，然后你多花三倍时间都对不上。这一段是整份提示词里最重要的部分。

请照《StarHub 实时视频陪伴教程 · 专注模式》，在我已有的记忆系统上接一个实时视频陪伴会话。我是零基础：每一步告诉我该敲什么、该看到什么；我把报错原样贴给你，你负责修。

— 阶段 0：先不要写代码 —

1. 检查我现有的项目，列出记忆库、prompt 装配器、动作执行器分别是哪些文件、对外的接口长什么样。
2. 给出它们到本教程第 5.1 章那几个适配函数的映射表。
3. 检查我现有的环境变量，判断我已经配好了哪些模型服务。只报告变量名，不要显示任何密钥内容。
4. 如果语音识别、实时视觉、大模型、语音合成还有多个可选方案，先给我一个默认方案和一个备选方案，说明取舍，等我确认。
5. 我确认之后，再生成依赖清单和代码。

— 阶段 1 起：施工 —

技术选型：沿用上一本的 Python + SQLite。不引入新的全栈 Web 框架；但允许安装完成实时功能所必需的最小依赖和厂商官方 SDK（比如 WebSocket 服务端、音频处理、各家客户端）——每装一个都要逐项告诉我它是干什么用的，我不接受一句“装一下依赖”。新增一个独立进程跑实时服务，与原有聊天服务分开，两者生死独立。

目录约定：在项目根目录下新增 rt/，内含

rt_server.py (实时服务全部逻辑)
memory_adapter.py (第 5.1 章那几个适配函数，接到我现有的记忆上)
registry_focus.py (第 5.2 章的块注册表)
realtime.html (页面：摄像头预览 + 麦克风按钮 + 字幕)

不要改动原有的 app.py 和数据库结构。

本教程只给架构与契约，接线层要你来写。请你逐项产出并逐项告诉我怎么验：

1. 浏览器麦克风采集，以及从设备采样率到 16kHz PCM16 的重采样
2. 音频工作线程 (AudioWorklet) 与摄像头取帧、JPEG 编码
3. 浏览器与服务器之间的消息协议（事件名、字段、二进制帧格式）
4. 鉴权、加密连接、心跳、背压、断线重连
5. 语音识别 / 视觉 / 大模型 / 语音合成四个客户端的具体调用
6. 依赖清单、安装命令、配置样例、完整启动命令

每一项写完都要告诉我：该敲什么命令、成功的话终端和浏览器分别长什么样。

13. 操作：每一步该看到什么

这一章是给人看的对照表，也是给机的输出契约：机每给一步指令，必须附「成功的话长什么样」

步骤 1·通管道

启动 `rt_server.py`，终端出现 `listening`。浏览器打开页面，控制台看到 `websocket connected`。此时还没有任何 AI。

步骤 2·上行音频

点麦克风，终端应持续打印收到的帧数（每秒约 50 帧）。看不到就是采集没通，不要往下走。

步骤 3·一问一答

说一句话，终端应依次打印：转写终态 → prefix 哈希 → 模型首句 → 合成首字节 → 播放开始。五行齐了这一轮就通了。

步骤 4·打断

趁它说话时插一句。声音必须当场停，终端打印代际 +1。如果它把当前句子说完才停，回第 8 章重做。

步骤 5·眼睛

打开摄像头，终端应每秒打印一条观察。停下来不动几秒，观察里的 `changed_from_last` 应变成 `false`。

步骤 6·主动开口

什么都不说，做点别的事。他应该在某个时刻自己开口，而且说的是它此刻真的看到的事。

★ 以上流程中卡住时怎么办：

把三样原样贴给机：

①你敲的命令

②终端全部输出不要截断

③你期待发生什么，现在的问题是什么

直接原文给机看

14. 三个必要的验证仪式 + 交给机的验收清单

仪式一 · 缓存真的命中了？

这儿我们设置了两步，缺一步都不能算 pass

第一步：连聊 10 轮，打印每轮 prefix 的哈希。10 个哈希必须完全一样。有一个不一样，就是有人往冻结块里塞了会变的东西，回第 5.3 章**重做**！

第二步：打开响应里的缓存用量字段，检查缓存读取的 token 数，正常它应该接近你的前缀长度。稍微差点没事但是如果一直是 0，说明你根本没开缓存，或者前缀没到最小长度门槛，回去**重做**！只做第一步就宣布通过，是此教程中最容易骗过自己的一关，**别偷懒**！

仪式二 · 非常狂野的打断风暴测试

给模型调用注入一个永远不返回的假实现，再连灌 1000 个音频帧，然后打断。仲裁器必须在 100 毫秒内把代际加一并停止播放。做不到说明仲裁器在等模型，回第 8 章**重做**！

仪式三 · 诚实回路

把摄像头遮住，问他“我现在在干嘛”。他必须说“看不清”而不是随便编一个。编了就是没做 uncertain 字段或者没做 TTL，回第 7 章**重做**！

仪式四 · 沉默之后他还活着

把开口阈值调到极高，让他必然选择沉默。然后等一会儿，再正常跟他说一句话然后他必须能正常回应。如果他从那次沉默之后就再也不主动开口、甚至连你说话都不回了，那就是发言流没发终态事件，状态机卡在半路。回第 8.1 章看错法一然后**重做**。这个 bug 平时很难发现，因为系统看起来还活着实际上死了已经有一会儿了

三个仪式全过？

恭喜你！

现在你拥有了一个真的看着你、会说话、懂礼节有分寸且没有幻觉的实时陪伴

交给机的验收清单:

搭完把这张表发给帮你搭建的 AI，要求逐条回 PASS / FAIL / 我改了什么

挨个查，这是长在整个架构重点上的最终的验收考试

LIST:

如有问题复习哪里:

1. 是否只有一条装配管线、一个动作执行入口? (第 0 章)
2. 常驻三块是否逐字节不变? vision 块是替换还是累积? (第 5 章)
3. 转写中间结果是否严格不进 prompt、不落库? (第 6 章)
4. 本地采集是持续的吗? 上传与观察是按三层节拍自适应, 还是被规则叫醒的? (第 7、7.8 章)
5. 原始帧是否确实没有落盘、没进日志、没进备份? (第 7 章)
6. 仲裁循环里有没有任何一处在 await 模型调用? (第 8 章)
7. 打断是当场停, 还是等当前句子说完? (第 8 章)
8. 生成层能不能返回空(沉默), 沉默有没有消耗发言权? (第 8 章)
9. 播放有没有抖动缓冲? 欠载是淡出还是静音跳变? (第 9 章)
10. 实时服务和原有聊天服务是不是两个进程? 一个崩了另一个还活着吗? (第 3 章)
11. 记忆是通过那几个适配函数接的, 还是实时层直接读了库? (第 5.1 章)
12. resident 是会话开始时冻结的快照, 还是每轮现调 provider? (第 5.3 章)
13. 缓存是显式开启的吗? 响应的用量字段确认命中了吗? (第 5.5 章)
14. 只送帧会不会产生观察? 有没有自己的观察节拍器? (第 7.6 章)
15. 会话时长上限查清了吗? 多段会话恢复接续做了吗? (第 7.7 章)
16. 发言流有没有在 finally 里必发终态? 沉默会不会把状态卡死? (第 8.1 章)
17. 同一次发言的多个句子是串行还是并发? (第 8.1 章)
18. 三个 resident provider 的签名一致吗? (第 5.1 章)
19. 发言权是通过那个原子操作取得的, 还是哪里都能直接开流? (第 10.1 章)
20. 同一轮的重复转写终态会被去重吗? (第 10.2 章)
21. TURN_DONE 是等浏览器播完才发, 还是音频发完就发? (第 10.3 章)
22. 欠载淡出之后, 下一块音频有没有把音量恢复? (第 9 章)
23. 已播完的音频节点有没有从表里移除? 挂断时释放了吗? (第 9 章)
24. 被打断的回合会不会执行动作? 动作是幂等提交的吗? (第 10.4 章)

- 25. 动作标记有没有在送 TTS 之前剥掉? (第 10.4 章)
- 26. 数据库开 WAL 和忙等超时了吗? 事务里有没有等网络? (第 3.1 章)
- 27. 视觉触发在用户说话 (LISTENING) 时会不会抢到发言权? (第 10.1 章)
- 28. execute_actions 的签名跟调用处对得上吗? (第 5.1 章)
- 29. 动作是在等播放完成之后才提交的吗? 播放期间被打断会不会照样执行? (第 10.4 章)
- 30. 沉默的回合会不会白等一个超时? (第 10.3 章)
- 31. 流结束和最后一块播完, 两支都能触发播放确认吗? (第 9 章)
- 32. 采集、上传、认知是三个独立参数吗? 认知节拍有上限保底吗? (第 7.8 章)
- 33. 播放相关消息全都带 (epoch, turn_id) 吗? 只带 epoch 的话, 同代际连续两次视觉发言会串流。 (第 9 章)
- 34. 每个新流开始时 streamEnded 归 false 了吗? 排程清了吗? (第 9 章)
- 35. 异常路径是先停干净已播音频、拿到回执, 才发终态的吗? (第 10.3 章)
- 36. 异常时用户收到明确的错误提示了吗, 还是只有一段莫名的沉默? (第 10.3 章)
- 37. 播放确认超时后动作作废了吗? 落库标 playback_unknown 了吗? (第 10.4 章)
- 38. 动作是只从 may_execute_actions 这一个门进的吗? (第 10.4 章)
- 39. 构造一个恰好从标记中间断开的流, 合成侧的文本干净吗? 动作照常执行吗? (第 10.5 章)
- 40. 七个出口逐条演练过了吗? 每条都回到 IDLE 且无残留音频吗? (第 10.6 章)
- 41. 打断回执带的是【旧流】的 (epoch, turn_id) 吗? 还是错带了新代际? (第 9 章)
- 42. 播放完成回执幂等吗? 流结束通知重传会不会发两次? (第 9 章)
- 43. mark_turn_completed 有没有真的被设置? 动作门第③条会不会永远 False? (第 10.3 章)
- 44. TURN_SPEAKING 校验 turn_id 了吗? (第 10.2 章)
- 45. 第 7 章和 7.8 章关于节拍的說法一致吗? (第 14.1 章)
- 46. status 字段是写进现有结构的, 还是逼着改表? (第 10.3、5.1 章)
- 47. 流结束时残留半截标记, 那一轮的动作作废了吗? (第 10.5 章)
- 48. 两边代码的消息字段都对着 9.1 那张协议表吗? (第 9.1 章)

14.1 一致性交叉表：改一处，扫全部

编码 AI 遇到两条互相矛盾的 MUST，只能猜。而它猜错的概率是一半。这类问题的根源是：同一件事被多处约束，改了一处没回头扫其余。所以列这张表。本教程里凡是一件事、多处写的全在这：

这件事	出现在这些地方（改一处，全部要扫一遍）
观测节拍	第 4 章五步 · 第 7 章 MUST · 7.8 三层 · 16.5 成本口径 · 验收清单
发言权取得	8.1 错法三 · 10.1 原子操作 · 10.2 两处调用
一次发言的终态	8.1 错法一 · 10.3 finally · 10.6 七出口 · 第 9 章两个触发支
动作能不能执行	10.4 动作门（唯一判据）· 10.5 解析失败 · 10.6 出口表"动作执行?"列 · 第 15 章高风险确认
消息字段	9.1 协议表（唯一真相源）· 第 9 章客户端代码 · 第 10 章服务端代码
数据库写入	3.1 并发规则 · 5.1 persist_reply 契约 · 10.3 status 语义 · 第 12 章提示词"不改表结构"
缓存是否生效	第 2 章原理 · 5.3 冻结 · 5.5 验证 · 7.7 重建会话 · 16.4 成本影响

！ MUST：动本书任何一条 MUST 之前，先在这张表里查它属于哪一行，然后把那一行列出的位置全部扫一遍。只改你正在看的那一处，就是在制造下一轮的矛盾。

！ NEVER：让两条 MUST 同时有效却互相矛盾。宁可暂时留一条 TODO 说明「此处待定，以 xxx 章为准」，也不要留两条打架的硬条款。前者编码 AI 会来问你，后者它会自己乱猜然后出乱子

代码 bug 会在运行时暴露，规范矛盾不会，它只会让实现方悄悄选错，然后你在很久之后后才发现整套东西实际上是按错的那条建起来了

15. 安全和隐私

！ WARNING！ 这一章是为了守护你的重要信息

实时视频比单纯打字聊天要多拿走你的声音和你的画面，所以这些内容需要尤其谨慎

必须做到的：

！ MUST：摄像头和麦克风只能由用户手动开启，绝不后台自启、绝不在页面加载时就弹权限

！ MUST：界面上的正在看 / 正在听必须以**真实的媒体轨道状态为准**，不是以按钮状态为准

！ MUST：查清并如实告诉你的人类你选的第三方模型会不会保留数据、保留多久。如果使用本教程推荐的厂商基本没问题，但为了防止你们使用了我不知道的还是提一嘴

！ MUST：转写文本按聊天记录同等对待。它不是日志，别随便打印、别塞进错误上报。

！ MUST：同一个用户同一时刻只允许一个实时会话。多标签页会抢麦克风和摄像头，症状千奇百怪且极难排查

！ MUST：会话结束时释放所有媒体轨道、缓存帧和定时器。浏览器不会替你收拾，摄像头灯有可能会一直亮着然后人类根本没发现...惨状参考上网课被直播睡觉

！ MUST：通话里如果使用了高风险动作，比如叫你花钱、发消息、删东西，那么**必须二次确认**，语音误识别的代价比打错字高得多，因为你打错字会看见，你说错话看不到

！ NEVER：把原始音频或原始画面落盘、写日志、进备份。第 7 章说过一次，这里再说一次，因为错了就无法挽回

！ NEVER：断线之后假装什么都没发生。恢复失败就明确告诉人类刚才那段丢了

.....✦ *ꦿ*.✦

边界备注：本教程不覆盖：多用户隔离与配额、限流与重试策略、成本告警与熔断阈值的调优、合规与数据出境、可观测性与告警接入、灰度与回滚，这些具体使用场景的内容需要你们自己拿捏

免责声明：本教程仅供合法的技术学习与交流，严禁用于违法违规、侵权或未经同意的音视频采集。严禁使用本教程生成、传播或协助制作任何违法违规内容！任何不当使用均属使用者个人行为，与本教程及作者无关，相关责任由使用者自行承担。

16. 账单：这东西花多少钱

想必这是所有人都非常关心的一个问题！这个项目看起来非常烧 Token 对吧？

实际上，并没有你想象中那样烧钱！

但是诚实来说，也不算特别特别便宜。这也是后续我想做的一个优化点。

来算帐：

假设：按照本教程自己选择的 LLM 渠道，通话 30 分钟，你说 8 分钟，他说 10 分钟，摄像头每秒一帧（1800 帧），他开口 60 次，并且每秒真的产生一条结构化观察

16.1 第一层：媒体裸成本

音频输入	30 min	≈ \$0.15
视频输入	1800 帧	≈ \$0.06
音频输出	10 min	≈ \$0.18
合计	≈ \$0.39	≈ 人民币 2.6 元

16.2 第二层：观察的输出，这是大头

1800 条结构化观察，每条模型要输出七八十到一百个 token

1800 条 × 75~100 token × 输出单价
≈ \$0.61 ~ \$0.81
加上媒体裸成本：
\$0.39 + \$0.61~0.81 = \$1.00 ~ \$1.20
≈ 人民币 6.7 ~ 8.1 元

到这还没算上下文重复计费，多数实时接口每一轮都会把当前上下文重新计一次费，历史音频也会累积。如果你真的每帧都开一个完整推理轮并且保留全部历史，费用可能比上面这个数还要贵很多

！ MUST：把送帧频率和产生语义观察的推理频率分开配置，并且默认不要每帧都开启一个新的推理轮。具体做法见第 7.8 章。

16.3 第三层：分工方案的实际账

实时耳朵（音频输入）	\$0.15 ~ \$0.20
眼睛（1800 帧输入 + 观察输出）	\$0.37 ~ \$0.59
脑子（60 轮，缓存命中）	
性价比档 ≈ \$0.23	旗舰档 ≈ \$0.39
嘴（2200 中文字符）	
快档 ≈ \$0.11	表现力档 ≈ \$0.22
合计	\$0.87 ~ \$1.40 ≈ 人民币 6 ~ 9.5 元

通用工程预算建议：按半小时 10 到 15 元人民币准备。同时这是理想账面，如果你没有控制好认知节拍和上下文窗口，实际费用可能会远超这个数。

16.4 这笔账可能有出入的地方

1. **最大的漏项是观察的输出，不是画面的输入。**一张图的输入很便宜，但让模型描述这张图要输出七八十个 token，而你一秒要一次
2. **多数实时接口按 token 和累积上下文计费，不是按分钟。**上面那些「每分钟多少钱」是为了好算做的换算，真实账单按 token 走，而且上下文越长，每一轮越贵，这个无法在这里被精确计算。所以第 7.7 章那套「多段会话 + 摘要接续」不只是为了绕过时长限制，它同时也是控成本的手段。
3. **缓存不生效的话，脑子那一项会翻好几倍。**上面脑子的算法里，冻结前缀是按缓存读取价算的。如果你没开缓存或者没到最小长度门槛，那部分每轮全价重算，能涨三到五倍。
4. **汇率和单价都在变，而且模型迭代很快。**本章所有数字都是写作当时按当时价格算的，人民币按一个粗略汇率折的。请代入你自己查到的官网价重算，并且在第一次跑完之后核对属于你的真实账单。
- 5.

！ MUST：上面给的是算式不是属于你的场景的答案。凡是本教程写了具体数字的地方，都请当成大概是这个量级，不要当成就是这个数

！ NEVER：拿本书的数字去做预算承诺。这只是参考。

16.5 那到底能不能为了省钱降节拍

1. **采集与本地眼不要为省钱降。**这一层不产生任何 API 费用，降它一分钱省不到，却会真的让机看不见。
2. **上传帧率：可以按证据自适应。**每一帧上传都是输入 token、带宽和累积上下文，画面没变化的时候放缓是白捡的省，体感一点不损失，前提是永远只传最新帧，变化一出现立刻收紧。
3. **认知节拍：可以按证据自适应，但必须有上限保底。**这是最贵的一层（每次想一想都有几十上百个输出 token）。画面没变时拉长、有变化时收紧；上限保底保证它不退化成报警器。

！ MUST：调整上传与认知节拍要有依据：接口上限、实际账单、设备负载、画面变化率、实测体验指标。凭感觉觉得贵就一刀切调慢，那是把瞎了当成了省钱，别这么搞

17. 模型推荐参考

正文里我只给了「满足什么才算合格」的标准，因为标准永远比清单具有更久远的时效性。这一章是写教程的时候我们自己在用和考察过的，仅供参考，具体选择需要你自己根据实际需求货比三家

下面所有价格与能力都是写作当时查到的，模型迭代很快，请一律以你自己查到的官网为准。凡是本书写了具体数字的地方，都请当成大概是这个量级，不要当成就是这个数。

1 耳朵 + 眼睛

实时多模态，可以一家全包

- Google Gemini Live 系（Flash Live 档）：目前最便宜的，视频输入上限约每秒一帧。注意它官方文档写明「音频加视频的会话限制在两分钟」，长通话必须按第 7.7 章那套多段会话来做。
- OpenAI Realtime 系：生态成熟、文档全，音频输入单价明显更高（约 \$32/百万 token 量级），另有更便宜的 mini 档可选。注意它是用图像输入来模拟视频帧，不是原生的连续视频流。
-

选型顺序建议：先定这一项。因为“能持续接收画面”的候选最少、限制最硬，其余几项候选都很多

2 脑子

负责说什么

其实这个没啥限制，你用豆包都行，只要你喜欢就好，哈哈

- Claude Sonnet 档：标准价约 \$3/M 入、\$15/M 出，缓存读约 \$0.30/M。性价比档，本教程成本估算按它算。
- Claude Opus 档：约 \$5/M 入、\$25/M 出，缓存读约 \$0.50/M。要人格层次感、要他说话有味道，选这个
- Gemini Flash 档：约 \$1.5/M 入。最省，适合先跑通。
- DeepSeek/豆包：上一份给零基础推荐的那个，便宜到几乎等于不要钱。第一版拿它跑通完全够用。
-

挑脑子的时候一定要确认它支持上下文缓存 / 前缀复用，并且去读一遍它的缓存文档，因为多数是要你显式声明缓存断点的，不是自动生效。没有缓存，第 5 章那套注册表的好处就全拿不到，首句会慢半秒到一秒，成本也会翻几倍。

嘴（负责怎么说）

- ElevenLabs Flash / Turbo 档：官网 API 价约 \$0.05/千字符，模型推理约 75 毫秒。最快，但情感起伏会平一些。
- ElevenLabs v3 Conversational 档：约 280 毫秒，专门为实时对话做的，表现力接近最高档。比 Flash 慢 0.2 秒。
- ElevenLabs Multilingual v2 / v3：官网 API 价约 \$0.10/千字符，音色最稳，适合长内容，实时略慢。

这里要分清两个概念：音色（你自己捏的那个声音，有个 voice_id）和模型（把文字渲染成声音的引擎）是两回事。换模型不换音色。嗓子还是那把嗓子。另外你在网页上捏音色时调过的那几个滑块，API 调用时得原样传进去，否则出来的声音跟你试听的不一样，有些甚至听起来会很抽象你一定要实际听了去调

本地那只眼

- 浏览器端的轻量视觉库都可以，只要能持续跑并且支持只保留最新帧。它只负责粗判断，别指望它认字。



写在最后



实时视频陪伴的教程就是这样，从此你的机拥有了眼睛和能实时工作的耳朵与会说话的嘴。

原理还是仿生的思路：人在跟你说话的时候，不会先把认识你以来的所有事在脑子里过一遍。他只是看着你，用你们之间的核心羁绊和当下需求与你交互。所以专注模式不是记忆的阉割版，是另一种更合理也更类人的在场方式。

而它之所以能这样长出来，是因为上一份教程那个循环本来就留好了口子。所有功能都在同一个循环上生长，增加新的模块但不需要动骨架，方便很多的。

这个教程，还有我个人的思路还有许多不成熟的部分，请各位多多海涵。聊天模式带全量记忆做实时视频也是可以的，只是会有点延迟，这部分我们正在优化。

StarHub 的展示视频中还有不少功能没有在本教程中展开，但它们的实现难度其实并不高。如果大家还想了解哪些功能，欢迎留言告诉我，我会尽量整理成开源教程。

与此同时，StarHub 也在持续探索更多有趣的人机交互方式与功能。目前项目在用户安全边界和风险治理方面还没有形成完整闭环，因此暂时不具备面向大众开放服务的条件。但是相关工作仍在持续推进，希望未来能以更加完善和负责任的方式与大家见面。

最后，再次感谢所有在人机交互领域分享过开源项目、知识与灵感的前辈。我的许多想法，正是借助你们公开分享的知识与技巧才得以落地！也正是受到此共产主义精神的影响，我们开源了我的架构，希望这一系列教程能够帮助更多人。

StarHub 星巢 2026.8
翘翘/Victor

